

大作业常见问题解析（FAQ） V1.3

配套《大作业-软件安装指南V1.1》与《大作业题目-学生版V1.2》。

版本：V1.3 日期：2026-06-09

本文整理开发过程中高频出现的问题与解决方案，供学生自查。

V1.3 变更：修正 Q6/Q10/Q12 中 dpHex 值与物理位映射的一致性；Q10 增加非时间场景 dp 独立占位示例；Q12 修正 FORMAT RIGHT 后 dpHex 需重新计算。

V1.2.1 变更：修正 Q7 滚动规则与 Q12 的冲突（年份不滚动）；Q7 增补S3.3 "8 位"计算方式、dp 策略对流水判断的影响、小数点随方向跟随规则。

V1.2 变更：新增 Q12 时间/日期显示与小数点关系；新增 Q13 NTP 同步状态指示解读；原 Q12 改为 Q14。

V1.1 变更：新增 Q11 LED 接管模式切换规则。

Q1. 为什么必须用 Python 3.11.9?

现象

安装 `pyqt5-tools` 时报错：

```
ERROR: Could not find a version that satisfies the requirement pyqt5-tools
```

或 `pip install PyQt5` 成功但 `pyuic5` 命令不存在。

原因

`pyqt5-tools`（含 Qt Designer、`pyuic5` 等）的维护者 **仅发布到 cp311**（即 Python 3.11.x）。3.12 及以上版本无对应 wheel，pip 无法安装。

Python 版本	PyQt5 本体	pyqt5-tools / Qt Designer	结论
3.11.x	✓	✓	推荐
3.12.x	✓	X（无 wheel）	不可用
3.13+	部分兼容	X	不可用
3.10 及以下	✓	✓	可用但不推荐

解决方案

下载并安装 **Python 3.11.9**（`python-3.11.9-amd64.exe`，已放入 `software/` 目录）。
安装时勾选 **Add Python to PATH**，安装完成后验证：

```
python --version
# 应输出：Python 3.11.9
```

Q2. 工作目录如何命名？

推荐方案（折中）

开发阶段用英文短路径，提交时按规范重命名打包。

C:\dev\hw26-1234\	← 开发目录（1234 为学号后 4 位）
├─ mcu\	← S800 板工程
└─ pc_host\	← PC 上位机工程

命名雷区

雷区	示例	后果
中文路径	C:\作业\大作业\	Keil / Python / Git 均可能报奇怪错误
空格	C:\my project\	命令行工具路径解析异常
特殊符号	C:\hw(2026)\	部分工具将 () 当元字符
路径过深	C:\Users\xxx\Desktop\课程\嵌入式\26年\大作业\v2\	Windows 260 字符限制
系统保护目录	C:\Program Files\	无写入权限

提交时重命名

假设学号 567890123456，姓名"交小电"
将开发目录拷贝并重命名为规范格式：
Copy-Item -Recurse C:\dev\hw26-1234 C:\dev\大作业567890123456-交小电
然后压缩提交

Q3. 如何创建和进入 Python 虚拟环境（venv）？

三步操作

第一步：创建虚拟环境（只需执行一次）
cd C:\dev\hw26-1234\pc_host
python -m venv .venv
第二步：激活虚拟环境（每次打开新终端都要执行）
.\.venv\Scripts\Activate.ps1
第三步：验证（提示符前出现 (.venv) 即成功）
(.venv) PS C:\dev\hw26-1234\pc_host> python --version
Python 3.11.9

常见错误

错误现象	原因	解决
无法加载文件 ...Activate.ps1, 因为在此系统上禁止运行脚本	PowerShell 执行策略限制	运行 Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
激活后 python 仍指向全局	终端类型不对或未正确激活	确认提示符前有 (.venv)
python -m venv .venv 报错	当前 Python 非 3.11.9	用完整路径 C:\...\python.exe -m venv .venv

关键理解

venv 不是 "Python 的子目录"

venv 是一个 **独立的隔离沙箱**:

- 新建时只有 pip 本身, 没有任何第三方包
- 所有依赖必须在激活后手动安装
- 好处: 不同项目可以使用不同版本的库, 互不干扰

Q4. 激活 venv 后 import 报错 ModuleNotFoundError

现象

```
(.venv) PS> python main.py
ModuleNotFoundError: No module named 'PyQt5'
```

原因

新创建的 venv 是空的——只有 pip 和 setuptools, 不会继承全局已安装的包。

解决方案

```
# 确认已激活 venv (提示符有 .venv)
(.venv) PS> pip install -r requirements.txt
```

如果还没有 requirements.txt, 先手动安装核心依赖:

```
pip install PyQt5==5.15.9 pyqt5-tools pyserial ntplib requests astral matplotlib
```

安装完成后验证:

```
(.venv) PS> python -c "from PyQt5.Qtwidgets import QApplication; print('OK')"
OK
(.venv) PS> pyuic5 --version
Python User Interface Compiler 5.15.9 for Qt version 5.15.2
```

Q5. PyQt5 程序如何运行?

三种运行层次

层次 1: 纯代码 Hello World (验证环境)

```
# hello_qt.py
import sys
from PyQt5.QtWidgets import QApplication, QLabel

app = QApplication(sys.argv)
label = QLabel("Hello PyQt5!")
label.show()
sys.exit(app.exec_())
```

运行: `python hello_qt.py`

层次 2: Qt Designer + pyuic5 工作流 (推荐)

1. 打开 Qt Designer 设计界面 → 保存为 `main_window.ui`
2. 转换为 Python 代码:
`pyuic5 ui/main_window.ui -o ui_main_window.py`
3. 在 `main.py` 中导入并使用:

```
# main.py
import sys
from PyQt5.QtWidgets import QApplication, QMainWindow
from ui_main_window import Ui_MainWindow

class MainWindow(QMainWindow, Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)

if __name__ == '__main__':
    app = QApplication(sys.argv)
    win = MainWindow()
    win.show()
    sys.exit(app.exec_())
```

层次 3: 集成串口 (大作业最终形态)

```
# 在 MainWindow.__init__ 中启动后台串口线程
self.serial_worker = SerialWorker()
self.serial_worker.line_received.connect(self.on_line_received)
self.serial_worker.start()
```

Qt Designer 打开方式

```
# 激活 venv 后
(.venv) PS> designer
# 或者完整路径
(.venv) PS> .\venv\Lib\site-packages\qt5_applications\Qt\bin\designer.exe
```

Q6. *EVT:DISP 报文格式说明

格式定义 (V1.2 §5.3)

```
*EVT:DISP <8字符> <dpHex>
```

字段	规则
<8字符>	定长 8 位，按当前显示内容排列，空位用 <code> </code> 填充， 不含小数点
分隔符	一个 空格
<dpHex>	2 位十六进制，表示 8 个 digit 的小数点（dp 段）状态位图。bit N 对应第 N+1 个 digit（从左开始）： 1 =dp 段点亮， 0 =熄灭。本 FAQ 示例采用 bit0=最左，学生实际实现可按硬件设计调整，只要自洽即可

位图展开示例

以 dpHex = `0A` 为例：

十六进制	<code>0A</code>	
二进制	<code>0 0 0 0 1 0 1 0</code>	
digit序号	<code>8 7 6 5 4 3 2 1</code>	← 从左数第几个
对应 bit	<code>7 6 5 4 3 2 1 0</code>	
dp 状态	<code>灭 灭 灭 灭 亮 灭 亮 灭</code>	

→ 第 2 个和第 4 个 digit 的 dp 段点亮（小数点可见），其余熄灭。

注意 bit 方向：从左到右写出 dp 状态 `01010000` 直觉上像 `0x50`，但因为 bit0 对应最左边的 digit（LSB 在左），实际编码需翻转为 `00001010` = `0x0A`。简言之：数码管从左到右的顺序对应二进制低位到高位。

EVT:DISP 完整示例

当前显示时间 `12:30:45`（dp 附着，6 位数字 + 2 位空位）：

```
*EVT:DISP 123045__ 0A
```

- 8 字符：`1 2 3 0 4 5` （ 表示尾部空位，无字符显示）
- dpHex = `0A`：第 2 个 digit（显示 `2`）和第 4 个 digit（显示 `0`）dp 段点亮，视觉效果为 `12.30.45`

注意：小数点不计入 8 字符，信息全部编码在 dpHex 中。

Q7. 数码管滚动显示行为规范

问题 1：不超过 8 位需要滚动吗？

学生版 §3.3 原文："当显示内容多于 8 位时按固定速率流水"

结论：≤ 8 位不滚动，> 8 位自动滚动。

显示内容	位数	是否滚动
时间 12.30.45	6	不滚动
日期 26.05.25	6	不滚动
年份 2026.0525	8	不滚动
天气短显 26°C SUN	8	不滚动

问题 2：需要设置滚动启停键吗？

不需要。流水是自动触发的：

- 内容 > 8 位 → 自动进入流水模式
- 内容 ≤ 8 位 → 静态显示

按键职责：

按键	功能	不是
SPEED	流水速度切换（250ms / 500ms）	滚动开/关
FORMAT	流水方向切换（LEFT / RIGHT）	滚动开/关

问题 3：上位机发送的 *SET:MSG 文本显示多久？

行为规则（学生版 §3.3 + §5.2 + 开发要求 §12）：

退出条件	说明
>8 位自动退出	文本流水完整一遍后自动返回时钟显示
>8 位手动退出	流水期间按任意按键（除编辑键）立即中断回到时钟
≤8 位静态显示	显示 2-3 秒后自动返回时钟（学生版未明确规定，建议值）

不需要设置固定显示时长（如"显示 5 秒后退出"），>8 位退出机制是"走完一遍 + 按键中断"双保险。

完整行为对照表

场景	是否滚动	退出方式
时间/日期/年份 (≤8 位)	不滚动	—
*SET:MSG HELLO WORLD (12 位)	自动滚动	走完一遍 / 按键中断
*SET:MSG HI (2 位)	不滚动	静态显示 2-3 秒后返回时钟
*EVT:DISP 心跳更新	不改变滚动状态	保持当前模式

问题 4：§3.3 的“8 位”怎么算？

“多于 8 位时流水”中的“位”指数码管物理位置数（占用几个 digit），与小数点的处理方式直接相关：

dp 策略	小数点对位数的影响	示例
dp 附着（不占位）	不增加 digit 数	3.14 → 3 个 digit
dp 独立占位	每个点占 1 个 digit	3.14 → 4 个 digit

dp 附着场景（时间/日期，强制）

内容	字符数	dp	占 digit 数	流水？
HH.MM.SS → 123045	6	2 dp 附着	6	否
YY.MM.DD → 260525	6	2 dp 附着	6	否
YYYY.MMDD → 20260525	8	1 dp 附着	8	否

dp 独立占位场景（其他，可选）

内容	字符+点	占 digit 数	流水？
*SET:MSG 3.14	3+1	4	否
*SET:MSG A.B.C.D.E	5+4	9	是
*SET:MSG 192.168.1.1	8+3	11	是

问题 5：“小数点随方向跟随”是什么意思？

§3.3 原文：“小数点随方向跟随：左→右用本位小数点，右→左用下一位小数点”

此规则仅在流水（>8 位）时生效，描述内容在数码管窗口中滑动时 dp 段如何跟随：

FORMAT 方向	dp 规则	含义
LEFT（左→右）	用本位 dp	dp 跟着它所属的字符一起移动

FORMAT 方向	dp 规则	含义
RIGHT（右→左）	用下一位 dp	dp 视觉上附着在右邻字符的左边

此规则与“附着/占位”策略无关——无论哪种方式，流水中 dp 的跟随逻辑相同。

总结：小数点策略对流水的影响

维度	dp 附着（不占位）	dp 独立占位
时间/日期	强制使用	禁止（否则 YYYY.MMDD 超 8 位会滚动）
其他场景	可选	可选
对流水判断的影响	总位数更少，更不容易触发流水	总位数更多，更容易触发流水
对 *EVT:DISP	无影响（协议层解耦）	无影响
流水中 dp 跟随	适用	适用

验收清单

- ☐ 时间/日期/年份 均在 8 位内静态显示，不滚动
- ☐ *SET:MSG 超 8 位时自动流水
- ☐ 流水期间秒走时不停
- ☐ 按任意键（除编辑键）立即中断消息返回时钟
- ☐ 消息 ≤ 8 位时静态显示 2-3 秒后返回时钟
- ☐ 流水方向切换后小数点正确跟随（LEFT 本位 / RIGHT 下一位）

Q8. 闹钟行为速查

项目	V1.2 规定
触发条件	时、分、秒同时相等
响铃方式	节奏式（响-停-响），200ms 响 / 200ms 停
最大持续	≤ 10 秒自动停止
手动停止	FUNC 键立即停止
贪睡功能	无（V1.2 不要求）
优先级	响铃期间 FUNC 优先于编辑模式切换
LED 指示	使能=常亮，响铃中=快闪
上报	*EVT:ALARM（开始） / *EVT:ALARM_OFF（停止）

Q9. 按键映射速查（10 项）

NAME	物理位置	短按功能
<code>FUNC</code>	K1 (I2C)	编辑模式循环切换；响铃中关闹钟
<code>SHIFT</code>	K2 (I2C)	编辑中切换高亮字段
<code>ADD</code>	K3 (I2C)	当前字段 +1
<code>SAVE</code>	K4 (I2C)	保存并退出编辑
<code>DISP</code>	K5 (I2C)	主显示切换：时间/日期/年份
<code>SPEED</code>	K6 (I2C)	流水速度 2 级切换
<code>FORMAT</code>	K7 (I2C)	流水方向切换 LEFT/RIGHT
<code>EXT</code>	K8 (I2C)	预留，仅触发 <code>*EVT:KEY EXT</code>
<code>USER1</code>	GPIO	请求 PC 对时
<code>USER2</code>	GPIO	短显天气 5 秒

协议中 `*SET:KEY` 与 `*EVT:KEY` 的 `<NAME>` 取值即以上 10 项。
`*SET:KEY` 模拟执行后不上报 `*EVT:KEY`，避免 PC↔板环回。

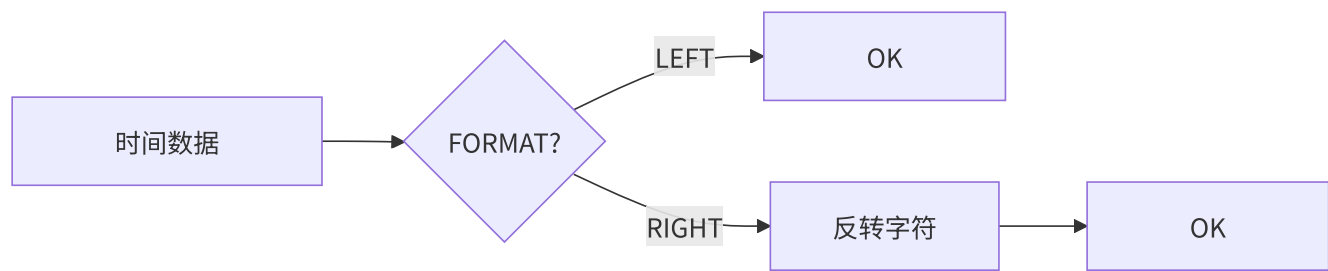
Q10. FORMAT RIGHT 的真正含义与 GET 应答逆序

问题背景

很多学生疑惑："为什么 `*SET:FORMAT RIGHT` 后，`*GET:TIME` 应答也要逆序？"

核心逻辑

`FORMAT` 控制的是 **S800 板端的数据输出方向**，而不仅仅是数码管显示方向。



为什么 GET 也要逆序？

原因	说明
协议一致性	<code>FORMAT</code> 是全局状态，影响所有输出方向（数码管 + <code>*GET</code> 应答 + <code>*EVT:DISP</code> ）
数字孪生对齐	PC 端收到 <code>*GET</code> 应答后，须与数码管显示一致，否则会出现"PC 文本显示 12.30.45 但面板显示 54.03.21"
硬件镜像语义	<code>FORMAT RIGHT</code> 的含义是"从右往左读"，所以人类阅读 PC 应答文本时，也应从右往左看

完整示例拆解

设当前时间 `12:30:45`（6 位数字，dp 附着在第 2、4 个 digit，时间/日期场景强制附着）：

LEFT 模式

数码管显示：[1] [2.] [3] [0.] [4] [5] [] []
 ↑dp ↑dp
`*GET:TIME` 应答：OK 12.30.45
`*EVT:DISP`: 123045__ 0A

- `dpHex 0A` = `00001010`₂ → 第 2、4 个 digit 的 dp 亮（附着在显示 2 和 0 的 digit 上）

RIGHT 模式（关键变化）

数码管显示：[] [] [5] [4.] [0] [3.] [2] [1]
 ↑dp ↑dp
`*GET:TIME` 应答：OK 54.03.21 ← 同步逆序
`*EVT:DISP`: __540321 28 ← 8 字符逆序，dpHex 重新计算

- `dpHex 28` = `00101000`₂ → 第 4、6 个 digit 的 dp 亮（逆序后 dp 跟随字符移动到新位置）

小数点跟随规则（核心难点）

方向	小数点规则	说明
<code>FMT_LEFT</code>	小数点在 本位	dp 附着在它所属的字符所在 digit 上
<code>FMT_RIGHT</code>	小数点在 下一位	逆序后 dp 必须“提前一位”才能在视觉上保持分隔位置

FORMAT 切换后，`dpHex` 需重新计算（不是简单保持不变）。

7SEG 物理位映射（时间/日期场景，dp 附着）

LEFT 模式（本 FAQ 示例采用 bit0=最左）：

[第1][第2][第3][第4][第5][第6][第7][第8]

[1][2][3][0][4][5][_][_]

↑dp↑dp

dp 附着在第 2、4 个 digit → bit1+bit3 = dpHex 0x0A

RIGHT 模式：

[第1][第2][第3][第4][第5][第6][第7][第8]

[_][_][5][4][0][3][2][1]

↑dp↑dp

dp 附着在第 4、6 个 digit → bit3+bit5 = dpHex 0x28

非时间/日期场景（dp 独立占位）的 FORMAT 示例

若学生在其他场景选择小数点独立占位（如 *SET:MSG 3.14，. 占一个 digit）：

LEFT 模式：

[第1][第2][第3][第4][第5][第6][第7][第8]

[3][.][1][4][_][_][_][_]

“.” 作为独立字符占第 2 个 digit，dpHex = 00（无 dp 段亮，点用段码画）

*EVT:DISP: 3.14__ 00

RIGHT 模式：

[第1][第2][第3][第4][第5][第6][第7][第8]

[_][_][_][_][4][1][.][3]

整个内容逆序，“.” 仍作为独立字符跟随移动

*EVT:DISP: __41.3 00

注：dp 独立占位时，小数点用段码（如 0x80 或自定义编码）显示在该 digit 上，dpHex 常为 00；“.” 作为普通字符参与逆序，不需要特殊处理。

验收清单

- ☐ *GET:TIME 在 RIGHT 模式下输出完全逆序（数字 + 小数点位置）
- ☐ *GET:DATE 同理逆序
- ☐ 数码管显示与 *GET 应答内容一致
- ☐ 小数点在 LEFT/RIGHT 切换时视觉位置不变（只是物理位变了）
- ☐ *EVT:DISP 的 8 字符部分同步逆序

Q11. *SET:LED 接管模式与正常模式如何切换？

问题背景

MCU 的 8 位 LED 默认由固件自主管理（心跳、闹钟、编辑、UART 等），但 PC 可通过 *SET:LED <hex2> 直接控制全部 LED。两者存在冲突：接管后原有指示如何恢复？

核心机制：两态模式

模式	LED 由谁控制	心跳/闹钟/编辑等指示
正常模式（默认）	MCU 自主管理	LED0-3 必做指示正常运作，LED4-7 扩展指示按需运作
接管模式	PC 通过 *SET:LED 直控	全部暂停，8 位 LED 完全由 PC 下发的 hex 值决定

进入接管

- 收到 *SET:LED <非00 的 hex2> → 立即进入接管模式

退出接管（三选一）

退出条件	说明
*SET:LED 00	PC 显式将 LED 全灭，视为退出接管
*RST	全局复位，恢复所有默认状态（学生版 §5.2 明确）
10 秒超时（自主增强）	10 秒内未收到新的 *SET:LED 则自动退出，属加分项

退出后恢复

- 退出接管后，所有 LED 默认含义立即恢复为当前实际状态（心跳继续闪、闹钟状态不丢失）
- 与天气/NTP 的冲突处理逻辑相同：接管时统一覆盖，退出后恢复

*EVT:LED 上报

无论正常模式还是接管模式，*EVT:LED <hex2> 都应如实上报当前 LED 实际输出值，确保 PC 数字孪生面板始终正确镜像。

验收清单

- ☐ *SET:LED 5A 后 LED 显示对应图案，心跳停止
- ☐ *SET:LED 00 后心跳恢复、闹钟/编辑等指示恢复
- ☐ *RST 后 LED 接管退出，恢复正常
- ☐ 接管期间 *EVT:LED 如实上报当前值

Q12. 时间/日期显示与小数点的关系

问题

小数点在数码管上是否占独立一位？时间 `HH.MM.SS` 和日期 `YYYY.MMDD` 能否在 8 位内完整显示？

答案

学生版未明确规定小数点是“独立占位”还是“附着在 digit 上”。但本作业做如下强制与开放：

层面	规则
协议层	小数点永远独立于 8 字符（ <code><dpHex></code> 单独编码），与显示方式无关
显示层（时间/日期）	强制 dp 附着不占位，保证 8 位内静态显示
显示层（其他）	附着或占位均可，学生自行决定

时间/日期占位对照

显示内容	占用数码管位数	说明
时间 <code>12.30.45</code>	6 位	dp 附着在第 2、4 个 digit，剩余 2 位填 <code> </code> 即空位
日期 <code>26.05.25</code>	6 位	dp 附着在第 2、4 个 digit，剩余 2 位填 <code> </code> 即空位
年份 <code>2026.0525</code>	8 位	dp 附着在第 4 个 digit，刚好占满

对串口通讯的影响

关心点	结论
小数点是否影响 8 字符定长？	不影响 。小数点从不计入 8 字符
PC 端解析是否需区分显示类型？	不需要 。统一用 8 字符画段码 + dpHex 画小数点
FORMAT RIGHT 是否影响编码？	8 字符内容逆序，dpHex 随位置重新计算（详见 Q10）

验收清单

- ☐ 时间 `12.30.45` 上报为 `123045__ 0A`，小数点不占字符位
- ☐ 日期 `26.05.25` 上报为 `260525__ 0A`
- ☐ 年份 `2026.0525` 上报为 `20260525 08`
- ☐ 三种格式均在 8 位内静态显示，不触发滚动

Q13. NTP 同步状态指示 (§17) 如何理解？

问题

开发要求 §17 提到 NTP 同步状态机和长按 USER1 查询显示 `n.SY.xx`，具体含义是什么？

答案

状态机（LED7 指示）

状态	LED7 表现	进入条件
UNSYNCED	灭	上电默认，从未被 NTP 校准
SYNCED	常亮	PC 完成 NTP 对时并下发 <code>*SET:TIME</code> 成功
DRIFT	1Hz 慢闪	距上次对时超过 24 小时

再次对时可从任意状态回到 SYNCED。

长按 USER1 查询（数码管短显 3 秒）

显示格式：`n.SY.xx`

字段	含义	取值示例
<code>n</code>	距上次对时的小时数（0-9，溢出显示 9）	<code>5</code> = 5 小时前对过时
<code>SY</code>	固定标识（Sync）	始终为 <code>SY</code>
<code>xx</code>	当前状态缩写	<code>OK</code> = SYNCED / <code>NO</code> = UNSYNCED / <code>DR</code> = DRIFT

显示示例

数码管显示	含义
<code>5.SY.OK</code>	5 小时前对时，当前 SYNCED
<code>0.SY.OK</code>	刚完成对时，当前 SYNCED
<code>_ .SY.NO</code>	从未对时（n 无意义用 <code>_</code> 占位），UNSYNCED
<code>9.SY.DR</code>	超过 9 小时未对时，已 DRIFT

注：此处 `.` 属于"其他短显"场景，小数点是附着还是占位由学生自行决定（参见 Q12）。

Q14. 其他易错点汇总

序号	问题	正确做法
1	Git 版本过旧不支持新特性	安装 Git ≥ 2.40
2	pyuic5 命令不存在	确认在 venv 中且已安装 pyqt5-tools
3	串口助手与 Python 争抢 COM	关闭串口助手后再运行 Python 程序
4	*SET:MSG 内容被转大写	MCU 端整理函数遇 *SET:MSG 前缀须保留原文
5	FORMAT RIGHT 后 GET 应答忘记逆序	所有 *GET 应答字符串 + 数码管均须逆序
6	编辑状态 5 秒超时未实现	用软定时器计时，超时自动退出且不保存
7	venv 中 pip freeze 为空	正常现象，新 venv 需手动 pip install
8	PC 数字孪生未处理 NIGHT 模式	NIGHT 下仅点亮时分 4 位 + LED 仅心跳
9	USER1 按下后 PC 无反应	PC 须监听 *EVT:KEY USER1 并自动触发 NTP
10	requirements.txt 版本不锁定	建议 PyQt5==5.15.9，以 pip freeze 为准